



Slide 1

CS 170

Java Programming 1

Expressions

Duration: 00:00:41

Advance mode: Auto

Notes:

Hello there. This is the CS 170, Java Programming 1 lecture, Expressions

Now that you've learned to create numeric variables, it's time to turn your attention to the second part of the "primitive puzzle". You recall that primitives, unlike objects, are manipulated via operators, instead of methods. Such manipulations are called expressions, and are the topic of this lesson.

While expressions like those shown in this picture are cute, that's not exactly what we'll be talking about. What, exactly, do we mean by "an expression" in the context of a Java program? Let's take a look.

Expression Vocabulary

- What is an expression?

"Any combination of operators and operands which, when evaluated, yields a value"

- So, what are operators and operands?

Slide 2

Expression Vocabulary

Duration: 00:00:24

Advance mode: Auto

Notes:

Here's a formal definition of an expression.

An expression is any combination of operators and operands which, when evaluated, yields a value.

Well, that's not very helpful; what is an operand? What is an operator? What is evaluation? What does all this mean?

Operands

- Operands are symbols which represent values
- Here are some of the things that can act as operands

- Literals $3 + 5$ [3 and 5 are values]
- Variables $a - 3$ [a contains a value]
- Method call $2 * \text{func}(a)$ [func() produces a value]
- Expression $(a+3) * 5$ [a + 3 results in a value]

Slide 3

Operands

Duration: 00:00:58

Notes:

As you know, the first step is creating a Java program is writing the source code. When you create your source code, the symbols that indicate a value or storage location in memory are called operands.

Here are some things that can act as operands in your code:

- Literals, such as 3 and 5 are, actually or literally, values
- Variables, such as a in this example, are locations containing a value
- Calling a method, such as func(a) in this case, produces a value that can be used in a calculation.
- A sub-expression, such as (a + 3) is evaluated and results in a value, which can then be used in the rest of

Advance mode: Auto

the calculation.

The important thing to see is that **all** operands are just different ways of specifying **values** or **data**.

Operators

- Operators are symbols that **perform an operation**
 - Operation may be performed on one or more operands
- Every operator has these three characteristics:
 - **Arity**: number of operands required
 - Unary, binary, tertiary (ternary)
 - **Precedence**: order in which operations are performed
 - **Associativity**: determines whether operations should occur left-right or right-left

Slide 4

Operators

Duration: 00:01:37

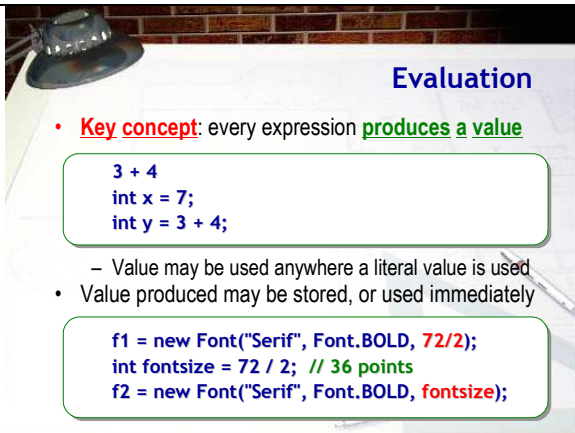
Advance mode: Auto

Notes:

In addition to operands, your programs will also contain operators. An operator is a symbol that is used to perform a calculation on one or more operands and to subsequently, produce a value.

Operators have three important characteristics:

- The first of this is called **arity**: arity determines the number of operands that the operator needs to carry out its work. Some operators require only a single operand; we call these unary operators. Most operators require two operands; we call these binary operators. Finally, one Java operator requires three operands; the ternary, (or tertiary), conditional operator.
- The **precedence** of an operator helps determine the order in which operations involving multiple operators are performed. Operators are placed in precedence groups, from highest to lowest. Operations that have higher precedence are performed before operations that have lower precedence.
- Finally, sometimes precedence isn't enough to determine exactly in what order the operators should be applied, and to which operands. In those cases, we fall back on the tie-breaking characteristic called associativity. Associativity determines whether operations involving multiple operators at the same level of precedence should proceed from right-to-left, (called right-associative), or from left-to-right, (called left-associative).



Evaluation

- **Key concept:** every expression **produces a value**

```

3 + 4
int x = 7;
int y = 3 + 4;

```

- Value may be used anywhere a literal value is used
- Value produced may be stored, or used immediately

```

f1 = new Font("Serif", Font.BOLD, 72/2);
int fontsize = 72 / 2; // 36 points
f2 = new Font("Serif", Font.BOLD, fontsize);

```

Slide 5

Evaluation

Duration: 00:01:21

Advance mode: Auto

Notes:

Now that we have operators and operands defined, let's finish our definition by examining the **key concept**: every expression produces a value when the expression is evaluated. The expression is evaluated when the line containing the expression is executed.

During evaluation, the operators and operands are combined and each operator is applied to its operands. The operator does its work and a temporary value is calculated and stored in memory.

Here's an example expression:

`3 + 4`

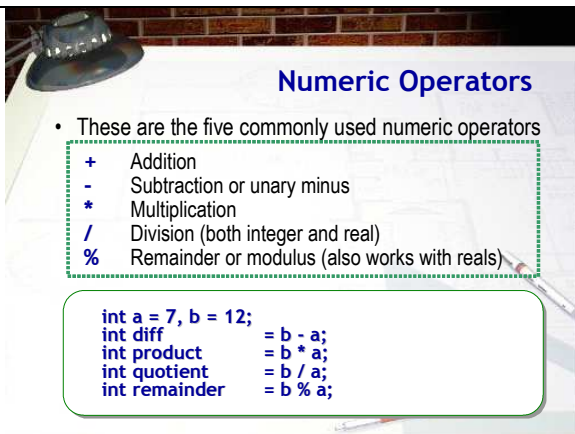
This expression uses two literal operands as well as the addition (or binary plus) operator. This expression produces or creates the temporary value 7.

The value produced by the expression can be used wherever a literal value can be used. For instance, both of these declarations store identical values into their respective variables:

`int x = 7;`

`int y = 3 + 4;`

Much of the time, a temporary value is all you need. If you need to store the result of an expression, however, you can use Java's assignment operator, (which you'll look at more closely in the next lesson), to copy the temporary value into a variable.



Numeric Operators

- These are the five commonly used numeric operators

+	Addition
-	Subtraction or unary minus
*	Multiplication
/	Division (both integer and real)
%	Remainder or modulus (also works with reals)

```

int a = 7, b = 12;
int diff    = b - a;
int product = b * a;
int quotient = b / a;
int remainder = b % a;

```

Slide 6

Numeric Operators

Duration: 00:01:30

Advance mode: Auto

Notes:

As you've already seen, Java has five arithmetic operators that work on both integers and floating point numbers (or a combination of the two).

- The plus sign represents the binary **addition** operator
- The minus sign does double duty. If you give it two operands (one on either side), it is the **subtraction** operator, but if you place it in front of a single literal or variable, it is the **unary minus** operator.
- **Multiplication** is performed by the "star" operator, rather than the x of mathematics
- **Division**, both real and integer, is performed by the forward slash
- The percent sign is used, not as a percentage operator, but to calculate the **remainder** of a division, again with both integers and reals.

Most of the five operators are self-explanatory, although you might find the remainder and division operators a little tricky if you've never programmed before.

Here are some integer variables and numeric expressions. What values do you suppose each of the following variables will hold after each expression is evaluated?

As you might expect, the variable `diff` will contain the value 5 and the variable `product` will contain the value 84. But, how about the last two?



Slide 7

Integer Division Operations

Duration: 00:02:42

Advance mode: Auto

Notes:

In some computer languages, different symbols are used for different kinds of division (integer or real). In Java, the same symbol is used for both. If one of the operands is a floating-point number, then real division is performed. If both are integers, then integer division is carried out.

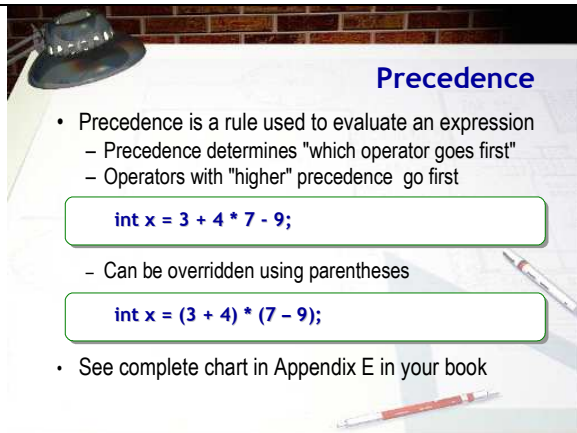
Integer division works like the division you learned in grade school. You draw a little house on the board and put the number you want to divide (called the dividend) inside the house. You draw the number you want to divide by (the divisor), standing at the front door of the house like a visitor. You then ask yourself, "how many visitors" could fit inside the house and place that number on the roof. This is the quotient. You multiply the quotient by the divisor, place the result underneath the dividend and subtract. The remainder is anything left over down in the basement, 8 in the example on the left and 3 in the example on the right.

When you perform integer division in Java, the quotient is calculated, but the remainder is discarded. It's important that you understand that when you divide two integers, the result is truncated, not rounded. With real division, $15/4$ would be 3.75 but with integer division, it's just 3, not 4 as it would be if the 3.75 was rounded.

The `%` or modulus operator does exactly the same thing, except, instead of returning the quotient portion from the roof it returns the remainder from the basement. If you were to divide 12 by 7, the quotient would be 1, since there is 1 7 in 12. $12 \% 7$, though, is 5 because that's what's left over after dividing 12 by 7.

The only tricky part about this is what happens if either the divisor or the dividend, (or both) are negative. Some languages prohibit the operation in that case, but in Java you just ignore the sign while calculating the result, and then the result is given the same sign as the numerator (that is, the dividend).

OK, now it's your turn. Create a new section in your in-class lab-exercise document. For Exercise 1, see if you can figure out what the values should be for `a`, `b`, `c`, and `d` in these expressions. After you've written down your answers, open up Code Pad and try them out. Shoot me a screen-shot of your Code Pad session.



Precedence

- Precedence is a rule used to evaluate an expression
 - Precedence determines "which operator goes first"
 - Operators with "higher" precedence go first

```
int x = 3 + 4 * 7 - 9;
```

- Can be overridden using parentheses

```
int x = (3 + 4) * (7 - 9);
```

- See complete chart in Appendix E in your book

Slide 8

Precedence

Duration: 00:01:59

Advance mode: Auto

Notes:

Earlier in this lesson, I mentioned that operators have the three characteristics of arity, precedence, and associativity. Let's look at what we mean by precedence.

Precedence is one of the rules that determine the order in which operators are applied to operands. Each operator is given a particular priority. Those operators with a higher priority, or precedence, get to perform their operations before those of lower status.

When faced with an expression involving multiple operations, you use the rules of precedence to decide "who goes first." Here's an example; what value does the variable `x` have after this line of code is evaluated? `int x = 3 + 4 * 7 - 9;`

There are obviously a couple different ways you could evaluate this expression. One way would be to go from left to right so that:

- `3 + 4` is 7
- `7 * 7` is 49
- `49 - 9` is 40

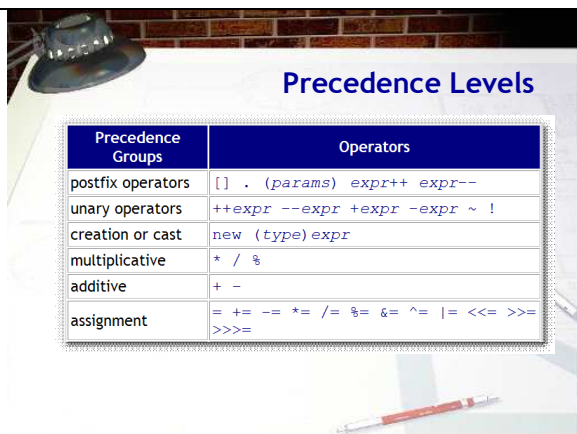
Applying the rules of precedence, however, you'll find that multiplication (`*`) has higher precedence, so you perform that first, followed by the addition and subtraction:

- `4 * 7` is 28
- `28 + 3 - 9` is 22

Quite a difference!

You can use parentheses to override the natural precedence in any way you like. For instance, doing this would result in `7 * -2` or `-14` for the answer.

You can find the complete precedence chart in Appendix E in your textbook but let me show you the short-hand version that will handle 95% of your programming needs.



Precedence Levels

Precedence Groups	Operators
postfix operators	<code>[] . (params) expr++ expr--</code>
unary operators	<code>++expr --expr +expr -expr ~ !</code>
creation or cast	<code>new (type) expr</code>
multiplicative	<code>* / %</code>
additive	<code>+ -</code>
assignment	<code>= += -= *= /= %= &= ^= = <<= >>=</code>

Slide 9

Precedence Levels

Notes:

Here's an abbreviated chart. It's still a little confusing when you first look at it, but I want to point out the operators we've used so far.

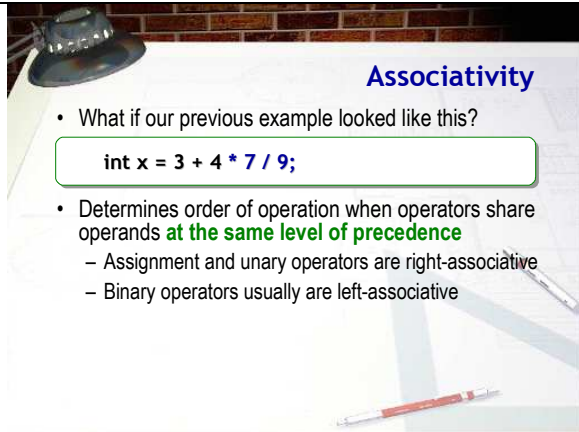
The highest three groups are the postfix, unary and creation groups. In our case, that means that the dot operator (used when you call a method) has the highest precedence. Unary minus is in the second group, while the cast operator along with the new operator are in the third. In the next lesson, we'll look at the increment and decrement operators which comprise the majority of the first two groups.

Other than unary minus, the arithmetic operators we've looked at today are all in either the multiplicative group: this includes multiplication, division and the remainder operator, or the

Duration: 00:01:18
Advance mode: Auto

additive group which consists of addition and subtraction.

Finally, at the very bottom (with quite a few groups not shown here) is assignment. Assignment is always at the bottom, because we want all other operations to be carried out before we store the finished result in a variable. If assignment were any higher, a partial result would be stored before the evaluation was completed.



Associativity

- What if our previous example looked like this?

```
int x = 3 + 4 * 7 / 9;
```
- Determines order of operation when operators share operands **at the same level of precedence**
 - Assignment and unary operators are right-associative
 - Binary operators usually are left-associative

Slide 10
Associativity
Duration: 00:01:25
Advance mode: Auto

Notes:

Suppose that our previous expression had read like this instead: `int x = 3 + 4 * 7 / 9;`

Now we're in a quandary. Both multiplication and division have the same precedence. But if we do the multiplication first we get this:

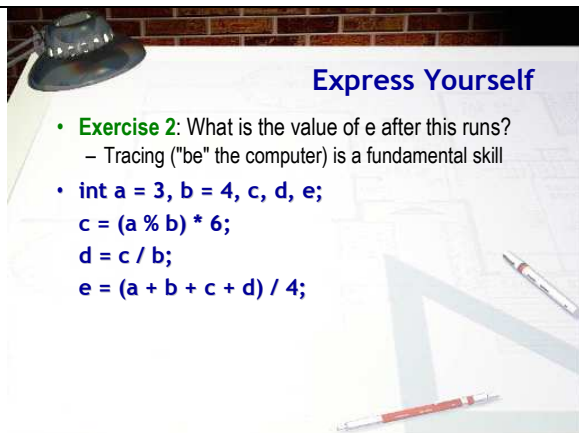
- $4 * 7$ is 28
- $28 / 9$ is 3
- $3 + 3$ is 6

If we do the division first, we get this:

- $7 / 9$ is 0 [Integer division]
- $4 * 0$ is 0
- $3 + 0$ is 3

So, which should we use? In situations like this, where precedence offers insufficient guidance, we have to fall back on the associativity of the operators. Associativity determines whether an operation is performed left-to-right or right-to-left and only comes into play when two operators share an operand, and both operators are at the same level of precedence.

The unary operators, postfix operators, and the assignment operators work right-to-left (they are right-associative). All of the other operators work left-to-right.



Express Yourself

- Exercise 2:** What is the value of `e` after this runs?
 - Tracing ("be" the computer) is a fundamental skill
- ```
int a = 3, b = 4, c, d, e;
c = (a % b) * 6;
d = c / b;
e = (a + b + c + d) / 4;
```

**Slide 11**

### Notes:

Let's wrap this lesson up with an exercise and a longer program. For Exercise 2, I want you to "be the computer." This is known as tracing, and is a fundamental skill you'll need to master if you want to be a good programmer.

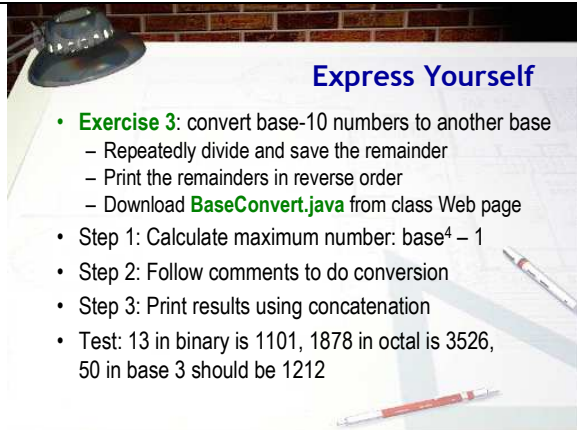
Simply create a column for each variable and mentally execute each line of code. As you execute the line of code, fill in the values underneath the columns. If you're using a Word processor, you can just add a table to your document and fill in the columns as you go.

When you're finished, use Code Pad to check your work, and shoot me a screen-shot of your Code Pad session.

## Express Yourself

Duration: 00:00:44

Advance mode: Auto



### Express Yourself

- **Exercise 3:** convert base-10 numbers to another base
  - Repeatedly divide and save the remainder
  - Print the remainders in reverse order
  - Download [BaseConvert.java](#) from class Web page
- Step 1: Calculate maximum number:  $\text{base}^4 - 1$
- Step 2: Follow comments to do conversion
- Step 3: Print results using concatenation
- Test: 13 in binary is 1101, 1878 in octal is 3526, 50 in base 3 should be 1212

## Slide 12

### Express Yourself

Duration: 00:04:22

Advance mode: Auto

## Notes:

To finish up this lecture, let's write a larger program that uses some computations.

One algorithm for converting a base 10 number to another base  $b$  involves repeatedly dividing by  $b$ . Each time a division is performed the remainder and quotient are saved. At each new step, the dividend is the quotient from the preceding step; the divisor is always  $b$ . The algorithm stops when the quotient is 0. The number in the new base is the sequence of remainders in reverse order (the last one computed goes first; the first one goes last).

In this exercise you will use this algorithm to write a program that converts a base 10 number to a 4-digit number in another base (you don't know enough programming yet to be able to convert any size number). The base 10 number and the new base (between 2 and 9) will be input to the program.

Download the starter code, `BaseConvert.java`, from the class Web page and store it in this week's BlueJ project. (You may have to close and re-open the project to see the file.)

Modify your program one step at a time as follows:

This program will only work correctly for base 10 numbers that fit in 4 digits in the new base. We know that in base 2 the maximum unsigned integer that will fit in 4 bits is 1111 binary which equals 15 in base 10 (or  $2^4 - 1$ ). In base 8, the maximum number is 7777 which equals 4095 in base 10 (or  $8^4 - 1$ ). In general, the maximum base 10 number that fits in 4 base  $b$  digits is  $b^4 - 1$ .

Add an assignment statement to the program to compute this value for the base that is input and assign it to the variable `maxNumber`. Then, add a statement that prints out the result (appropriately labeled) so that the users will know the maximum number that they can enter in any given base. Compile and run the program to make sure it is correct so far.

Now add the code to do the conversion. The comments below guide you through the calculations—replace them with the appropriate Java statements.

So far the program does not print out the answer. Recall that the answer is the sequence of remainders written in reverse order—note that this requires concatenating the four digits that have been computed. Since they are each integers, if we just add them the computer will perform arithmetic instead of

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p>concatenation. So, we will use a variable of type String.</p> <p>Note near the top of the program a variable named baseBNum has been declared as an object of type String and initialized to an empty string. Add statements to the program to concatenate the digits in the new base to baseBNum and then print the answer. Test it using the following values:</p> <ul style="list-style-type: none"><li>• Enter 2 for the base and 13 for the base 10 number—the program should print 1101 as the base 2 value.</li><li>• Enter 8 for the base and 1878 for the number—the program should print 3526 for the base 8 value.</li><li>• Enter 3 for the base and 50 for the number—the program should print 1212.</li></ul> <p>Shoot me two screen-shots for this problem. One of your code and one showing the three runs I've requested.</p> |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|